

Guia de Comandos Git

Guia de Comandos Git — Referência Rápida

1. Configuração inicial (só faz uma vez por computador)

```
git config --global user.name "Seu Nome"
```

Define o nome que aparece como autor dos seus commits.

```
git config --global user.email "seuemail@exemplo.com"
```

Define o e-mail vinculado aos seus commits (deve ser o mesmo cadastrado no GitHub).

```
git config --global init.defaultBranch main
```

Garante que toda pasta nova onde você rodar `git init` já crie a branch principal com o nome `main`.

```
git config --list
```

Lista todas as configurações do Git ativas (serve pra conferir se tudo foi salvo certo).

```
git config --global --list
```

Mostra só as configurações globais (as que valem pra todos os seus repositórios).

2. Chave SSH (autenticação com o GitHub)

```
ssh-keygen -t ed25519 -C "seuemail@exemplo.com"
```

Gera um novo par de chaves (privada + pública). A privada fica só no seu PC; a pública você cadastra no GitHub.

```
cat ~/.ssh/id_ed25519.pub
```

Mostra o conteúdo da chave pública no terminal, pra você copiar e colar no GitHub.

```
ssh -T git@github.com
```

Testa se a conexão SSH com o GitHub está funcionando. Resposta de sucesso: `Hi usuario! You've successfully authenticated...`

3. Criando ou clonando um repositório

```
git init
```

Transforma a pasta atual em um repositório Git novo (do zero).

```
git clone git@github.com:usuario/repositorio.git
```

Baixa um repositório que já existe no GitHub (com todo o histórico e branches), pra um computador novo.

4. Navegação no terminal (Git Bash)

```
pwd
```

Mostra o caminho completo de onde você está agora.

```
ls
```

Lista os arquivos e pastas do local atual.

```
cd nome-da-pasta
```

Entra em uma pasta. Use `Tab` pra autocompletar o nome.

```
cd ..
```

Volta uma pasta pra trás.

```
mkdir nome-da-pasta
```

Cria uma pasta nova.

```
touch arquivo.txt
```

Cria um arquivo vazio (Git Bash).

```
echo "texto" > arquivo.txt
```

Cria (ou sobrescreve) um arquivo já com conteúdo dentro.

```
echo "texto" >> arquivo.txt
```

Adiciona uma linha ao final do arquivo, sem apagar o que já tinha.

5. Dia a dia — salvando mudanças

```
git status
```

Mostra o que mudou desde o último commit (o comando mais usado de todos).

```
git add nome-do-arquivo.txt
```

Prepara um arquivo específico para ser salvo no próximo commit.

```
git add .
```

Prepara **todos** os arquivos modificados/novos da pasta atual.

```
git commit -m "mensagem descrevendo a mudança"
```

Salva permanentemente no histórico tudo que foi preparado com **add**.

```
git commit -am "mensagem"
```

Atalho: já adiciona e commita arquivos que o Git **já conhece** (não funciona pra arquivos totalmente novos).

```
git commit --amend
```

Corrige a mensagem (ou conteúdo) do último commit feito.

```
git mv nome-antigo.txt nome-novo.txt
```

Renomeia um arquivo já deixando a mudança preparada para commit.

6. Vendo o histórico

```
git log
```

Mostra o histórico completo de commits (autor, data, mensagem, hash).

```
git log --oneline
```

Versão resumida do histórico, uma linha por commit.

```
git log --oneline --graph --all
```

Mostra o histórico de forma visual, incluindo todas as branches — ótimo pra entender a “árvore” do projeto.

```
git log --follow nome-arquivo.txt
```

Mostra o histórico de um arquivo específico, mesmo que ele já tenha sido renomeado.

```
git diff
```

Mostra exatamente o que mudou (linha a linha) e ainda não foi commitado.

7. Branches (linhas do tempo paralelas)

```
git branch
```

Lista as branches locais (a atual aparece com ).

```
git branch -a
```

Lista também as branches remotas (que existem no GitHub).

```
git branch nome-da-branch
```

Cria uma branch nova, sem mudar pra ela.

```
git checkout nome-da-branch
```

Muda para uma branch já existente.

```
git checkout -b nome-da-branch
```

Cria uma branch nova e já muda pra ela (atalho mais usado).

```
git switch -c nome-da-branch
```

Versão mais moderna do comando acima (mesmo efeito).

```
git checkout -b nova-branch <hash-do-commit>
```

Cria uma branch nova a partir de um commit específico (não da branch atual).

```
git branch -m nome-antigo nome-novo
```

Renomeia uma branch (falha se já existir uma com esse nome).

```
git branch -M nome-antigo nome-novo
```

Renomeia forçadamente, mesmo se já existir uma branch com esse nome.

```
git branch -d nome-da-branch
```

Apaga uma branch (só permite se ela já foi mesclada em outro lugar).

```
git branch -D nome-da-branch
```

Apaga forçadamente, mesmo sem ter sido mesclada (útil pra abandonar experimentos).

8. Juntando branches (merge)

```
git checkout branch-que-vai-receber  
git merge outra-branch
```

Junta o conteúdo de `outra-branch` dentro da branch atual.

```
git merge nome-branch -m "mensagem do merge"
```

Faz o merge já definindo a mensagem, sem abrir o editor de texto (Vim).

```
git merge --abort
```

Cancela um merge em andamento (se algo deu errado no meio do caminho).

Se o Vim abrir pedindo mensagem de merge:

1. Aperte `Esc`
 2. Digite `:wq` e aperte `Enter` → salva e sai (aceita a mensagem)
 3. Ou digite `:q!` e aperte `Enter` → sai sem salvar
-

9. Trabalhando com o GitHub (remoto)

```
git remote add origin git@github.com:usuario/repositorio.git
```

Conecta seu repositório local a um repositório no GitHub, apelidado de `origin`.

```
git remote -v
```

Mostra quais repositórios remotos estão conectados.

```
git remote remove origin
```

Remove a conexão com o remoto (útil pra reconfigurar do zero se algo deu errado).

```
git push -u origin main
```

Envia commits pro GitHub pela primeira vez, já ligando sua branch local com a remota.

```
git push
```

Envia novos commits pro GitHub (usado depois do primeiro `push -u`).

```
git push -u origin nome-da-branch
```

Envia uma branch nova pro GitHub pela primeira vez.

```
git fetch origin
```

Busca as atualizações de todas as branches do GitHub, sem misturar nada nos seus arquivos ainda.

```
git merge origin/nome-da-branch
```

Junta na branch atual a versão mais recente de uma branch remota (depois de um `fetch`).

```
git pull
```

Atalho que já faz `fetch` + `merge` de uma vez, atualizando a branch atual com o que está no GitHub.

```
git pull origin main --allow-unrelated-histories
```

Usado quando local e remoto têm históricos diferentes (ex: GitHub criou README sozinho) e você precisa juntar os dois à força.

10. Desfazendo coisas

```
git restore nome-do-arquivo.txt
```

Descarta mudanças não commitadas em um arquivo, voltando ele pro estado do último commit.

```
git reset nome-do-arquivo.txt
```

Remove um arquivo do “stage” (do `add`), mas mantém a mudança no arquivo.

```
git checkout <hash-do-commit>
```

Visualiza temporariamente como o projeto estava naquele commit (sem alterar nada, modo “detached HEAD”).

```
git checkout <hash-do-commit> -- nome-arquivo.txt
```

Traz de volta a versão de **um arquivo específico** de um commit antigo.

```
git reset --soft <hash-do-commit>
```

Volta a branch pra um commit anterior, mas mantém as mudanças como não-commitadas (mais seguro).

```
git reset --hard <hash-do-commit>
```

⚠ Volta a branch pra um commit anterior, **apagando permanentemente** tudo que veio depois. Nunca use em commits já enviados ao GitHub.

```
git revert <hash-do-commit>
```

Cria um novo commit que desfaz as mudanças de um commit específico, mantendo todo o histórico visível. Forma mais segura de desfazer algo já compartilhado.

11. Extras úteis

```
git stash
```

Guarda temporariamente mudanças não commitadas (sem precisar commitar ainda), limpando a pasta de trabalho.

```
git stash pop
```

Traz de volta o que foi guardado com `stash`.

```
git tag nome-da-tag
```

Marca um commit importante, tipo uma versão (`v1.0`).

Legenda dos atalhos do terminal

Tecla / comando	O que faz
<code>Tab</code>	autocompleta nome de pasta/arquivo
<code>q</code>	sai de uma tela de paginação (ex: depois de <code>git diff</code> ou <code>git log</code>)
<code>Ctrl + C</code>	cancela um comando em execução

Fluxo típico do dia a dia (resumo geral)

```
git status           # ver o que mudou
git add .            # preparar tudo
git commit -m "mensagem" # salvar no histórico
git pull             # trazer atualizações do GitHub
git push             # enviar pro GitHub
```